

INFORMATION SECURITY LAB MANUAL



Dr. Muhammad Siddique

Information Security Lab Manual (Hands-On Cryptography, Network & Application Security)

Dr. Muhammad Siddique

August 2026

Laboratory Safety, Ethics, and Legal Notice

Every technique taught in this manual is presented for one purpose: to help you *defend* systems. Students must observe the following rules at all times.

- **Test only on systems you own or are explicitly authorised in writing to test.** All scanning, traffic capture, and vulnerability testing in this manual is to be performed inside the isolated classroom lab (virtual machines on a host-only network) and never against the campus network, an ISP, or any third-party system.
- **Unauthorised access is a criminal offence** under the Prevention of Electronic Crimes Act (PECA) 2016 in Pakistan and equivalent computer-misuse laws elsewhere. Curiosity is not a legal defence.
- **Handle real data responsibly.** Never capture, store, or share credentials, personal data, or traffic belonging to other people, including classmates.
- **Report responsibly.** If you discover a genuine vulnerability in a university system during your studies, report it privately to the department and do not disclose or exploit it.

Your instructor may require you to sign an acceptable-use agreement before the first lab session.

Contents

Laboratory Safety, Ethics, and Legal Notice	1
1 Week 1: Foundations of Information Security and Lab Setup	3
2 Week 2: Classical Ciphers and Cryptanalysis	6
3 Week 3: Symmetric Encryption – AES in Practice	10
4 Week 4: Asymmetric Cryptography and Key Exchange	14
5 Week 5: Cryptographic Hash Functions and Data Integrity	18
6 Week 6: Message Authentication Codes and Digital Signatures	22
7 Week 7: Password Security and Secure Credential Storage	26
8 Week 8: Authentication, Access Control, and Authorisation	30
9 Week 9: Network Security Fundamentals and Traffic Analysis	34
10 Week 10: Host Hardening, Firewalls, and Logging	38
11 Week 11: Malware Concepts and Defensive Analysis	41
12 Week 12: Web Application Security and Secure Coding	45
13 Week 13: Steganography, Data Protection, and Digital Forensics Basics	50
14 Week 14: Risk Management, Security Policy, and Incident Response	54

1 Week 1: Foundations of Information Security and Lab Setup

Objective: Understand the core goals, terminology, and threat landscape of information security, and build the isolated virtual laboratory environment used throughout this course.

Tasks:

1. **Task 1:** Explain the CIA triad (Confidentiality, Integrity, Availability) and give a concrete failure example of each.
2. **Task 2:** Distinguish between a threat, a vulnerability, a risk, and an exploit, and explain how they relate.
3. **Task 3:** Classify attacks as passive or active, and identify which CIA property each one violates.
4. **Task 4:** Set up an isolated virtual lab (a Linux VM and a second VM on a host-only network) and verify connectivity between them.
5. **Task 5:** Install and verify the Python cryptography toolchain used in later labs.

Details of Lab Experiment:

Task 1: The CIA triad

- **Confidentiality:** information is disclosed only to those authorised to see it. A failure looks like a database of customer records being read by an outsider – the data was not changed or destroyed, merely *seen* by the wrong party.
- **Integrity:** information is accurate and has not been altered by an unauthorised party. A failure looks like a bank balance or an exam result being silently modified; the data is still available and still confidential, but it is now *wrong*, which can be far more damaging than disclosure.
- **Availability:** information and services are accessible to authorised users when they need them. A failure looks like a denial-of-service condition or a ransomware incident that encrypts files – nothing leaked and nothing was falsified, but the organisation cannot operate.

A useful habit to build from day one: whenever you read about a security incident in the news, immediately ask "which leg of the triad broke?" Many real incidents break more than one, and identifying which ones tells you what controls were missing.

Task 2: Threat, vulnerability, risk, and exploit

- A **vulnerability** is a weakness in a system – unpatched software, a default password, an unvalidated input field.
- A **threat** is a potential cause of harm – a criminal group, a careless insider, a flood in the server room.
- An **exploit** is the actual technique or code that takes advantage of a specific vulnerability.
- **Risk** is the combination of likelihood and impact: informally, $\text{Risk} \approx \text{Threat} \times \text{Vulnerability} \times \text{Impact}$.

The practical consequence of this vocabulary is that you can reduce risk by acting on *any* of its factors: patching removes the vulnerability, network segmentation reduces the impact, and monitoring reduces the window in which a threat can act. Security work is almost always about reducing risk to an acceptable level, never about eliminating it entirely – a point you will formalise in Week 14.

Task 3: Passive versus active attacks

- **Passive attacks** observe without modifying: eavesdropping on unencrypted traffic, traffic analysis (learning *who* talks to *whom* and when, even if content is encrypted). They violate confidentiality and are difficult to detect, which is why the defence is *prevention* through encryption rather than detection.
- **Active attacks** modify data or system state: message tampering, replay, impersonation, denial of service. They violate integrity and/or availability, and because they leave traces, the defence combines prevention with *detection* (logging, integrity checks, intrusion detection).

Task 4: Building the isolated lab

Create two virtual machines in VirtualBox or VMware – for example an Ubuntu Server VM ("target") and an Ubuntu Desktop or Kali VM ("analyst") – and attach both to a **host-only** or **internal** network adapter so that no lab traffic can escape onto the campus network. Verify connectivity from the analyst VM:

```
# On each VM: find its address on the host-only network
ip addr show

# From the analyst VM, confirm the target is reachable
ping -c 4 192.168.56.101

# Confirm the VM cannot reach the outside world (expected:
  failure)
ping -c 2 8.8.8.8
```

Listing 1: Verifying isolated lab connectivity

That final check matters as much as the first: a lab that can still reach the internet is a lab in which a misconfigured tool can accidentally scan somebody else's network, which is precisely the situation the ethics notice at the front of this manual is designed to prevent.

Task 5: Installing the Python toolchain

```
python3 -m venv seclab
source seclab/bin/activate
pip install cryptography pycryptodome passlib pyotp
python3 -c "import cryptography; print(cryptography.
  __version__)"
```

Listing 2: Installing and verifying the cryptography toolchain

Note that we deliberately install a *maintained, audited* library rather than writing production cryptography by hand. You will implement several algorithms from scratch in Weeks 2–3 to understand how they work, but the professional rule – which this course will repeat often – is that hand-rolled cryptography in real systems is a vulnerability, not an achievement.

Summary:

This week you established the vocabulary of information security (CIA triad; threat, vulnerability, risk, exploit; passive versus active attacks) and built the isolated virtual laboratory and Python toolchain that every subsequent lab depends on. Keep the lab strictly isolated – every later exercise assumes it.

2 Week 2: Classical Ciphers and Cryptanalysis

Objective: Implement classical substitution and transposition ciphers, then break them, in order to understand why modern cryptography is designed the way it is.

Tasks:

1. **Task 1:** Implement the Caesar cipher for encryption and decryption.
2. **Task 2:** Break the Caesar cipher by brute force, and explain why its key space is fatally small.
3. **Task 3:** Implement the Vigenère cipher and explain how it resists simple frequency analysis.
4. **Task 4:** Perform frequency analysis on a monoalphabetic ciphertext and recover structure from it.
5. **Task 5:** Implement a columnar transposition cipher and contrast substitution with transposition.

Details of Lab Experiment:

Task 1: The Caesar cipher

```
def caesar(text, shift):
    result = ""
    for ch in text:
        if ch.isupper():
            result += chr((ord(ch) - 65 + shift) % 26 +
65)
        elif ch.islower():
            result += chr((ord(ch) - 97 + shift) % 26 +
97)
        else:
            result += ch          # punctuation and spaces
            pass through
    return result

plaintext = "Attack at dawn"
cipher = caesar(plaintext, 3)
print("Ciphertext:", cipher)
print("Decrypted :", caesar(cipher, -3))
```

Listing 3: Caesar cipher encryption and decryption

The Caesar cipher shifts every letter a fixed number of positions through the alphabet, so decryption is simply encryption with the negative shift – the modulo-26 arithmetic wraps Z back around to A. Notice that spaces and punctuation pass through unchanged, which is itself a weakness: it leaks the word lengths and sentence structure of the plaintext even before any letters are recovered.

Task 2: Brute-forcing the Caesar cipher

```
cipher = "Dwwdfn dw gdzq"

for key in range(1, 26):
    print(f"Key {key:2d}: {caesar(cipher, -key)}")
```

Listing 4: Exhaustive key search against a Caesar ciphertext

The entire key space is just 25 non-trivial keys, so a complete exhaustive search finishes instantly and a human can spot the meaningful English line immediately. This is the first and most important lesson of the course in concrete form: **a cipher is only as strong as its key space**, and secrecy of the algorithm ("security through obscurity") contributes nothing – a principle formalised as *Kerckhoffs's principle*, which holds that a cryptosystem must remain secure even when everything about it except the key is public knowledge.

Task 3: The Vigenère cipher

```
def vigenere(text, key, encrypt=True):
    result = ""
    key = key.lower()
    k = 0
    for ch in text:
        if ch.isalpha():
            shift = ord(key[k % len(key)]) - 97
            if not encrypt:
                shift = -shift
            base = 65 if ch.isupper() else 97
            result += chr((ord(ch) - base + shift) % 26 +
                           base)
            k += 1 # advance key only on
letters
        else:
            result += ch
    return result

ct = vigenere("Attack at dawn", "lemon", encrypt=True)
print("Ciphertext:", ct)
```



```
print("Decrypted :", vigenere(ct, "lemon", encrypt=False))
```

Listing 5: Vigenere cipher using a repeating keyword

Vigenère is *polyalphabetic*: each letter is shifted by a different amount depending on its position under the repeating keyword, so the same plaintext letter encrypts to different ciphertext letters at different positions. That flattens the letter-frequency distribution and defeats the naive frequency analysis of Task 4 – but it does not save the cipher, because the key repeats, and that repetition can be detected (the Kasiski examination and index-of-coincidence methods) to recover the key length and reduce the problem back to several independent Caesar ciphers.

Task 4: Frequency analysis

```
from collections import Counter

cipher = "Wkh txlfn eurzq ira mxpsv ryhu wkh odc b grj"
letters = [c.lower() for c in cipher if c.isalpha()]
total = len(letters)

for ch, count in Counter(letters).most_common(8):
    print(f"{ch}: {count:3d} ({100*count/total:5.2f}%)")
```

Listing 6: Letter frequency analysis of a ciphertext

In English, E is the most common letter (roughly 12%), followed by T, A, O, I, and N. Because a monoalphabetic substitution maps each plaintext letter to exactly one ciphertext letter, it *preserves* these frequencies – the shape of the distribution survives encryption even though the labels change, which is exactly the leak an analyst exploits. Run this on a few hundred characters of ciphertext rather than one sentence; frequency analysis is a statistical technique and needs enough data before the pattern is reliable.

Task 5: Columnar transposition

```
def transpose_encrypt(text, cols):
    text = text.replace(" ", "")
    rows = -(-len(text) // cols)          # ceiling
    padded = text.ljust(rows * cols, "X")
    out = ""
    for c in range(cols):
        for r in range(rows):
            out += padded[r * cols + c]
    return out
```

```
print(transpose_encrypt("Attack at dawn", 4))
```

Listing 7: Columnar transposition cipher

Transposition *rearranges* the letters rather than replacing them, so the ciphertext has exactly the same letter frequencies as the plaintext – frequency analysis reveals nothing about *which* letters map to which, but the telltale English-like distribution immediately tells an analyst that they are facing a transposition rather than a substitution. Modern block ciphers such as AES combine both ideas deliberately: repeated rounds of substitution (*confusion*) and permutation (*diffusion*), a design principle articulated by Claude Shannon and one you will see in action in Week 3.

Summary:

This week you implemented and broke classical ciphers, meeting Kerckhoffs's principle, key-space size, frequency analysis, and Shannon's confusion/diffusion ideas along the way. These ciphers have no place in real systems, but every weakness you exploited here explains a specific design decision in the modern algorithms that follow.

3 Week 3: Symmetric Encryption – AES in Practice

Objective: Use a modern symmetric block cipher correctly, understand block cipher modes of operation, and see why mode and IV choices matter as much as the algorithm itself.

Tasks:

1. **Task 1:** Explain symmetric encryption, block versus stream ciphers, and where AES fits.
2. **Task 2:** Encrypt and decrypt data using the high-level Fernet recipe.
3. **Task 3:** Use AES-CBC directly with an explicit key, IV, and padding.
4. **Task 4:** Demonstrate why ECB mode is unsafe by encrypting repeating data.
5. **Task 5:** Use AES-GCM to obtain confidentiality *and* integrity together.

Details of Lab Experiment:

Task 1: Symmetric encryption fundamentals

In symmetric encryption the same secret key both encrypts and decrypts, which makes it extremely fast – AES is implemented in hardware on virtually every modern CPU – but leaves the hard problem of getting that shared key to the other party safely, which is exactly what the asymmetric cryptography of Week 4 exists to solve. AES is a **block cipher**: it transforms fixed 128-bit blocks, using a 128-, 192-, or 256-bit key, over 10–14 rounds of substitution and permutation. A **stream cipher** (such as ChaCha20) instead generates a keystream that is XORed with the plaintext byte by byte.

Task 2: Fernet – the safe default

```
from cryptography.fernet import Fernet

key = Fernet.generate_key()           # 32 random bytes,
    base64-encoded
f = Fernet(key)

token = f.encrypt(b"Confidential: exam paper draft")
print("Token:", token)
```

```
print("Recovered:", f.decrypt(token))
```

Listing 8: Authenticated symmetric encryption with Fernet

Fernet is a *recipe*, not an algorithm: it bundles AES-128-CBC for confidentiality, an HMAC-SHA256 for integrity, a random IV, and a timestamp into one token, so that a developer cannot accidentally choose a broken mode or reuse an IV. When you need symmetric encryption in real work, reach for a recipe like this first; drop down to the lower-level primitives of Tasks 3–5 only when you have a specific reason and know exactly what you are doing.

Task 3: AES-CBC with explicit key, IV, and padding

```
import os
from cryptography.hazmat.primitives.ciphers import Cipher,
    algorithms, modes
from cryptography.hazmat.primitives import padding

key = os.urandom(32)                # AES-256 key
iv = os.urandom(16)                 # fresh random IV per
    message

padder = padding.PKCS7(128).padder()
data = padder.update(b"Attack at dawn") + padder.finalize()
    ()

encryptor = Cipher(algorithms.AES(key), modes.CBC(iv)).
    encryptor()
ct = encryptor.update(data) + encryptor.finalize()
print("Ciphertext:", ct.hex())

decryptor = Cipher(algorithms.AES(key), modes.CBC(iv)).
    decryptor()
padded_plain = decryptor.update(ct) + decryptor.finalize()
unpadder = padding.PKCS7(128).unpadder()
print("Plaintext:", unpadder.update(padded_plain) +
    unpadder.finalize())
```

Listing 9: AES in CBC mode with PKCS7 padding

Two details carry almost all the security weight here. First, the **IV must be random and unique for every message** under a given key – it is not secret and is normally transmitted alongside the ciphertext, but reusing it leaks relationships between messages. Second, AES only processes whole 128-bit blocks, so **padding** extends the final partial block; PKCS7 appends n bytes each of value n , which the receiver strips after decryption.

Task 4: Why ECB mode is unsafe

```
import os
from cryptography.hazmat.primitives.ciphers import Cipher,
    algorithms, modes

key = os.urandom(32)
# Three identical 16-byte blocks
data = b"YELLOW SUBMARINE" * 3

ecb = Cipher(algorithms.AES(key), modes.ECB()).encryptor()
ct = ecb.update(data) + ecb.finalize()

for i in range(0, len(ct), 16):
    print("Block", i // 16, ":", ct[i:i+16].hex())
```

Listing 10: Demonstrating that ECB leaks plaintext structure

Run this and observe that all three ciphertext blocks are *identical*. Because ECB encrypts each block independently with no chaining, identical plaintext blocks always produce identical ciphertext blocks, so the structure of the plaintext survives encryption – the classic demonstration is encrypting a bitmap image in ECB mode and still being able to make out the picture in the “encrypted” result. ECB should be treated as unusable for any real data; CBC or, better, an authenticated mode like GCM is the correct choice.

Task 5: AES-GCM – authenticated encryption

```
import os
from cryptography.hazmat.primitives.ciphers.aead import
    AESGCM

key = AESGCM.generate_key(bit_length=256)
aesgcm = AESGCM(key)
nonce = os.urandom(12)           # 96-bit nonce, never
    reused with this key

ct = aesgcm.encrypt(nonce, b"Transfer PKR 50,000", b"
    account-id-42")
print("Ciphertext:", ct.hex())
print("Plaintext :", aesgcm.decrypt(nonce, ct, b"account-
    id-42"))

# Tamper with one byte and observe that decryption FAILS
    rather than
# silently returning wrong data
tampered = bytearray(ct); tampered[0] ^= 0x01
try:
```

```
    aesgcm.decrypt(nonce, bytes(tampered), b"account-id-42")
except Exception as e:
    print("Tampering detected:", type(e).__name__)
```

Listing 11: AES-GCM providing confidentiality and integrity

CBC mode protects confidentiality but *not* integrity – an attacker who cannot read the message may still be able to flip bits in it in predictable ways. GCM is an **AEAD** (Authenticated Encryption with Associated Data) mode: it produces an authentication tag alongside the ciphertext, so any modification causes decryption to fail loudly, as the tampering test above demonstrates. The "associated data" argument (`account-id-42` here) is authenticated but not encrypted, which is useful for binding a ciphertext to its context so it cannot be replayed elsewhere.

Summary:

This week you used AES correctly through both a high-level recipe and the low-level primitives, saw concretely why ECB mode leaks structure, and learned why authenticated encryption (GCM) rather than plain confidentiality should be your default. The recurring lesson: with modern cryptography, the algorithm is rarely the weak point – the mode, the IV/nonce handling, and the key management are.

4 Week 4: Asymmetric Cryptography and Key Exchange

Objective: Understand public-key cryptography, generate and use RSA key pairs, and see how Diffie–Hellman solves the key-distribution problem that symmetric cryptography cannot.

Tasks:

1. **Task 1:** Explain the public/private key model and the key-distribution problem it solves.
2. **Task 2:** Generate an RSA key pair and serialise it to PEM files.
3. **Task 3:** Encrypt and decrypt a short message with RSA-OAEP.
4. **Task 4:** Implement Diffie–Hellman key exchange and verify both parties derive the same secret.
5. **Task 5:** Build a hybrid scheme: RSA to protect an AES key, AES to protect the data.

Details of Lab Experiment:

Task 1: The public-key model

Each participant holds a mathematically linked **key pair**: a *public* key that may be freely published and a *private* key that must never leave its owner. Anything encrypted with the public key can only be decrypted with the private key (giving confidentiality), and anything signed with the private key can be verified by anyone holding the public key (giving authenticity, which you will use in Week 6). This solves the problem symmetric cryptography cannot: two parties who have *never met* can communicate securely, because no secret ever has to travel between them.

The cost is speed. RSA is orders of magnitude slower than AES and can only encrypt data smaller than its key size, which is why real protocols never encrypt bulk data with RSA – they use the hybrid construction of Task 5.

Task 2: Generating an RSA key pair

```
from cryptography.hazmat.primitives.asymmetric import rsa
from cryptography.hazmat.primitives import serialization
```



```
private_key = rsa.generate_private_key(public_exponent
                                       =65537, key_size=2048)
public_key = private_key.public_key()

with open("private.pem", "wb") as f:
    f.write(private_key.private_bytes(
        encoding=serialization.Encoding.PEM,
        format=serialization.PrivateFormat.PKCS8,
        encryption_algorithm=serialization.
        BestAvailableEncryption(b"lab-passphrase")
    ))

with open("public.pem", "wb") as f:
    f.write(public_key.public_bytes(
        encoding=serialization.Encoding.PEM,
        format=serialization.PublicFormat.
        SubjectPublicKeyInfo
    ))

print("Key pair written to private.pem / public.pem")
```

Listing 12: Generating and saving an RSA key pair

Note that the private key file is itself encrypted with a passphrase – a private key sitting unprotected on disk is one stolen laptop away from compromising every message and signature it protects. Open both `.pem` files in a text editor afterwards and observe the difference: the public file is safe to email to anyone, while the private file is the crown jewel of the whole scheme.

Task 3: RSA encryption with OAEP padding

```
from cryptography.hazmat.primitives.asymmetric import
padding
from cryptography.hazmat.primitives import hashes

message = b"Session key material"

ciphertext = public_key.encrypt(
    message,
    padding.OAEP(mgf=padding.MGF1(algorithm=hashes.SHA256
    ()),
                algorithm=hashes.SHA256(), label=None)
)
print("Ciphertext length:", len(ciphertext))

recovered = private_key.decrypt(
    ciphertext,
    padding.OAEP(mgf=padding.MGF1(algorithm=hashes.SHA256
    ()),
```

```

        algorithm=hashes.SHA256(), label=None)
    )
    print("Recovered:", recovered)

```

Listing 13: RSA-OAEP encryption and decryption

The OAEP padding is not optional decoration. "Textbook" RSA – raw modular exponentiation with no padding – is deterministic, which means the same plaintext always yields the same ciphertext and an attacker can simply encrypt guesses and compare. OAEP introduces randomness and structure that defeats this and several other classes of attack, which is why every modern RSA deployment mandates a proper padding scheme.

Task 4: Diffie–Hellman key exchange

```

import secrets

# Small parameters for illustration ONLY; real DH uses
# 2048+ bit primes
p = 0xFFFFFFFFFFFFFFFFC90FDAA22168C234C4C6628B80DC1CD1
g = 2

a = secrets.randbelow(p - 2) + 1      # Alice's private
    value
b = secrets.randbelow(p - 2) + 1      # Bob's private value

A = pow(g, a, p)                      # Alice sends A
    publicly
B = pow(g, b, p)                      # Bob sends B
    publicly

alice_secret = pow(B, a, p)
bob_secret   = pow(A, b, p)

print("Shared secrets match:", alice_secret == bob_secret)

```

Listing 14: Diffie-Hellman shared secret derivation

Both parties arrive at the same value $g^{ab} \bmod p$ while only ever transmitting g^a and g^b ; an eavesdropper who sees both public values still faces the discrete logarithm problem, which is computationally infeasible for properly sized parameters. Critically, though, plain Diffie–Hellman authenticates nobody – an active attacker sitting in the middle can run a separate exchange with each party and relay traffic between them. That is exactly why real protocols (TLS, Week 7's signatures, Week 9's certificates) always combine key exchange with authentication.

Task 5: Hybrid encryption

```
import os
from cryptography.hazmat.primitives.ciphers.aead import
    AESGCM

# 1. Generate a fresh symmetric key for this message only
session_key = AESGCM.generate_key(bit_length=256)

# 2. Encrypt the bulk data quickly with AES-GCM
nonce = os.urandom(12)
bulk = AESGCM(session_key).encrypt(nonce, b"A very long
    report..." * 100, None)

# 3. Encrypt only the small session key with the recipient
    's RSA public key
wrapped_key = public_key.encrypt(
    session_key,
    padding.OAEP(mgf=padding.MGF1(algorithm=hashes.SHA256
    ()),
                    algorithm=hashes.SHA256(), label=None)
)

# --- Recipient side ---
unwrapped = private_key.decrypt(
    wrapped_key,
    padding.OAEP(mgf=padding.MGF1(algorithm=hashes.SHA256
    ()),
                    algorithm=hashes.SHA256(), label=None)
)
print("Decrypted OK:", AESGCM(unwrapped).decrypt(nonce,
    bulk, None)[:21])
```

Listing 15: Hybrid encryption: RSA-wrapped AES key

This hybrid pattern – slow asymmetric cryptography used only to protect a small, fresh symmetric key, which then does the fast bulk work – is precisely what TLS, PGP, and encrypted messaging apps all do internally. Understanding it means you understand the structural shape of nearly every real-world secure channel.

Summary:

This week you generated RSA keys, encrypted with OAEP, derived a shared secret with Diffie–Hellman, and combined asymmetric and symmetric cryptography into the hybrid scheme used by essentially every practical secure protocol. Remember the two limits you met: RSA needs padding, and unauthenticated key exchange is vulnerable to an active attacker in the middle.

5 Week 5: Cryptographic Hash Functions and Data Integrity

Objective: Use cryptographic hash functions to verify integrity, understand their required properties, and recognise which algorithms are obsolete.

Tasks:

1. **Task 1:** Compute hashes with several algorithms and compare their digest lengths.
2. **Task 2:** Demonstrate the avalanche effect by changing a single bit of input.
3. **Task 3:** Verify the integrity of a downloaded file against a published checksum.
4. **Task 4:** Explain preimage resistance, second-preimage resistance, and collision resistance.
5. **Task 5:** Build a simple tamper-evident hash chain and detect a modified record.

Details of Lab Experiment:

Task 1: Computing hashes

```
import hashlib

data = b"Information Security Lab"

for name in ["md5", "sha1", "sha256", "sha512", "sha3_256"]:
    h = hashlib.new(name, data).hexdigest()
    print(f"{name:9s} ({len(h)*4:3d} bits): {h}")
```

Listing 16: Comparing hash algorithms and digest sizes

A cryptographic hash maps input of any length to a fixed-length digest deterministically and irreversibly. Note the digest sizes: **MD5 (128-bit) and SHA-1 (160-bit) are cryptographically broken** – practical collisions have been demonstrated for both – and must never be used for signatures, certificates, or integrity guarantees against a motivated adversary. SHA-256 and the SHA-3 family are the current safe defaults; MD5 survives only as a non-security checksum for accidental corruption.

Task 2: The avalanche effect

```
import hashlib

a = hashlib.sha256(b"Attack at dawn").hexdigest()
b = hashlib.sha256(b"Attack at dusk").hexdigest()

print("A:", a)
print("B:", b)

# Count how many bits differ between the two digests
x = int(a, 16) ^ int(b, 16)
print("Bits differing:", bin(x).count("1"), "of 256")
```

Listing 17: A one-character change alters roughly half the digest bits

You should see roughly 128 of the 256 bits change – about half – even though the inputs differ by a few characters. This **avalanche effect** is essential: it means the digest reveals nothing about how similar two inputs were, so an attacker cannot inch toward a target hash by making small adjustments and watching the output get "closer."

Task 3: Verifying a download

```
import hashlib

def file_sha256(path):
    h = hashlib.sha256()
    with open(path, "rb") as f:
        for chunk in iter(lambda: f.read(8192), b''):
            h.update(chunk)
    # stream: never loads the whole file
    return h.hexdigest()

published = "e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855"

actual = file_sha256("ubuntu.iso")
print("MATCH" if actual == published else "MISMATCH - DO NOT INSTALL")
```

Listing 18: Hashing a file in chunks and comparing to a published checksum

Reading the file in 8 KB chunks matters for practical reasons – it lets you hash a multi-gigabyte ISO without loading it into memory. But note the limit of this control: a checksum published on the *same* compromised server as the file protects only against accidental corruption and transmission errors, not against a determined attacker. Genuine protection requires the checksum to be *signed* (Week 6) so its own authenticity can be verified.

Task 4: Required properties

- **Preimage resistance:** given a digest h , it is infeasible to find any input m with $\text{hash}(m) = h$. This is what makes stored password hashes non-reversible.
- **Second-preimage resistance:** given a specific input m_1 , it is infeasible to find a different m_2 with the same digest. This is what stops an attacker substituting a malicious file for a legitimate one with a matching checksum.
- **Collision resistance:** it is infeasible to find *any* two distinct inputs sharing a digest. This is the strongest requirement and the first to fall – it is precisely collision resistance that MD5 and SHA-1 have lost, which is why forged certificates became possible.

Note that collisions must exist mathematically (infinite inputs, finite outputs, by the pigeonhole principle); "resistance" only means finding one must be computationally infeasible.

Task 5: A tamper-evident hash chain

```
import hashlib

def build_chain(records):
    chain, prev = [], "0" * 64
    for r in records:
        digest = hashlib.sha256((prev + r).encode()).
        hexdigest()
        chain.append((r, prev, digest))
        prev = digest
    return chain

def verify(chain):
    prev = "0" * 64
    for record, stored_prev, digest in chain:
        if stored_prev != prev:
            return False
        if hashlib.sha256((prev + record).encode()).
        hexdigest() != digest:
            return False
        prev = digest
    return True

log = build_chain(["login:ali", "transfer:5000", "logout:
ali"])
print("Intact chain valid:", verify(log))
```

```
log[1] = ("transfer:500000", log[1][1], log[1][2])    #  
    tamper with a record  
print("After tampering :", verify(log))
```

Listing 19: Chaining hashes to make modification detectable

Because each entry's digest incorporates the previous entry's digest, altering any record invalidates every digest after it – an attacker must recompute the entire remainder of the chain to hide a change. This is the core mechanism behind tamper-evident audit logs and, extended with distributed consensus, behind blockchain ledgers.

Summary:

This week you computed and compared hashes, observed the avalanche effect, verified file integrity, learned the three resistance properties, and built a tamper-evident hash chain. The headline practical rule: **use SHA-256 or SHA-3; treat MD5 and SHA-1 as broken.**

6 Week 6: Message Authentication Codes and Digital Signatures

Objective: Distinguish integrity from authenticity, use HMAC for shared-secret authentication, and use digital signatures for non-repudiable authentication.

Tasks:

1. **Task 1:** Explain why a bare hash does not prove authenticity.
2. **Task 2:** Generate and verify an HMAC over a message.
3. **Task 3:** Use constant-time comparison and explain the timing-attack risk.
4. **Task 4:** Create and verify an RSA digital signature.
5. **Task 5:** Compare MACs and signatures, and explain non-repudiation.

Details of Lab Experiment:

Task 1: Why a hash alone is not enough

If a message travels with a plain SHA-256 digest, an attacker who can modify the message in transit can simply recompute the digest over their modified version – the hash function is public, so nothing stops them. A hash proves only that data has not been *accidentally* corrupted; proving that it came from a particular party and was not *deliberately* altered requires a secret to be involved, which is what MACs (shared secret) and signatures (private key) provide.

Task 2: HMAC

```
import hmac, hashlib, secrets

shared_key = secrets.token_bytes(32)      # both parties
      know this
message = b"Transfer PKR 50000 to account 12345"

tag = hmac.new(shared_key, message, hashlib.sha256).
      hexdigest()
print("MAC tag:", tag)

# --- Receiver side ---
expected = hmac.new(shared_key, message, hashlib.sha256).
      hexdigest()
```



```
print("Authentic:", hmac.compare_digest(tag, expected))

# An attacker who modifies the message cannot produce a
valid tag
forged = b"Transfer PKR 99999 to account 66666"
bad = hmac.new(b"guessed-key-wrong", forged, hashlib.
    sha256).hexdigest()
print("Forgery accepted:", hmac.compare_digest(bad,
    expected))
```

Listing 20: Generating and verifying an HMAC-SHA256

HMAC mixes the shared secret into the hashing process in a specific nested construction (rather than naively concatenating key and message, which is vulnerable to length-extension attacks against certain hash functions). Only a party holding the key can produce a valid tag, so a correct tag proves both integrity *and* that the sender knew the secret.

Task 3: Constant-time comparison

```
import hmac

a = "a3f5c9d1e7b2"
b = "a3f5c9d1e7b9"

print("Naive :", a == b) # returns
early on first mismatch
print("Safe :", hmac.compare_digest(a, b)) # always
scans both fully
```

Listing 21: Why constant-time comparison is used instead of ==

An ordinary == comparison of strings stops as soon as it finds a mismatched byte, so a wrong tag differing in the first byte is rejected measurably faster than one differing in the last. An attacker who can time many verification attempts can exploit this to recover a valid tag one byte at a time – a **timing side-channel attack**. `compare_digest` always examines the full length, removing the timing signal. Use it for every secret comparison: MAC tags, session tokens, API keys, and password hashes.

Task 4: RSA digital signatures

```
from cryptography.hazmat.primitives.asymmetric import rsa,
padding
from cryptography.hazmat.primitives import hashes
from cryptography.exceptions import InvalidSignature
```

```
private_key = rsa.generate_private_key(public_exponent
                                     =65537, key_size=2048)
public_key = private_key.public_key()

document = b"I approve the purchase order of PKR 250,000."

signature = private_key.sign(
    document,
    padding.PSS(mgf=padding.MGF1(hashes.SHA256()),
                  salt_length=padding.PSS.MAX_LENGTH),
    hashes.SHA256()
)

try:
    public_key.verify(document, signature,
                      padding.PSS(mgf=padding.MGF1(hashes.SHA256()),
                                      salt_length=padding.PSS.MAX_LENGTH),
                      hashes.SHA256())
    print("Signature valid - document is authentic and
          unmodified")
except InvalidSignature:
    print("Signature INVALID")
```

Listing 22: Creating and verifying an RSA-PSS signature

Signing hashes the document first and then applies the *private* key, so the signature is short regardless of document size, and verification with the *public* key succeeds only if both the signer's identity and the document contents are unchanged. Modify a single character of `document` before verifying and confirm the `InvalidSignature` path is taken.

Task 5: MACs versus signatures

- An **HMAC** uses one secret shared by both parties. It is fast and compact, but because *either* party could have produced any given tag, it cannot settle a dispute – the receiver could have forged the message themselves.
- A **digital signature** uses the signer's private key, which nobody else holds. This provides **non-repudiation**: the signer cannot credibly deny having signed, which is what makes signatures the basis of legally recognised electronic signatures, code signing, and the certificate system you will meet in Week 9.

Choose an HMAC when both endpoints are under your control and speed matters (API request authentication, session cookies); choose a signature when a third party must be able to verify authorship, or when the signer must be held accountable.

Summary:

This week you separated integrity from authenticity, produced and verified HMACs and RSA-PSS signatures, and met the timing side-channel that makes constant-time comparison mandatory. The distinguishing question to carry forward: *does a third party need to verify this, and must the signer be unable to deny it?* If yes, you need a signature, not a MAC.

7 Week 7: Password Security and Secure Credential Storage

Objective: Understand how passwords should be stored, why salting and key stretching are mandatory, and how to evaluate password strength.

Tasks:

1. **Task 1:** Explain why plaintext and plain-hash password storage both fail.
2. **Task 2:** Implement salted password hashing with PBKDF2.
3. **Task 3:** Use a modern password hash (bcrypt/argon2) via passlib.
4. **Task 4:** Measure how work factor affects verification time.
5. **Task 5:** Build a password strength checker and estimate entropy.

Details of Lab Experiment:

Task 1: Why naive storage fails

Storing passwords in **plaintext** means a single database disclosure hands the attacker every account immediately – and because users reuse passwords, it also compromises their accounts on unrelated services. Storing a **plain hash** (e.g. a bare SHA-256) is only marginally better: hashes are fast by design, so an attacker with the database can compute billions of guesses per second offline, and identical passwords produce identical hashes, which instantly reveals which users share a password and allows precomputed lookup tables to be applied to the whole database at once.

The two fixes are **salting** (a unique random value per user, making every hash unique and precomputed tables useless) and **key stretching** (deliberately slow, repeated hashing that makes each guess expensive).

Task 2: PBKDF2 with a per-user salt

```
import hashlib, os, hmac

def hash_password(password, iterations=600_000):
    salt = os.urandom(16)  # unique per user
    dk = hashlib.pbkdf2_hmac("sha256", password.encode(), salt, iterations)
    return salt.hex() + "$" + str(iterations) + "$" + dk.hex()
```

```
def verify_password(password, stored):
    salt_hex, iters, dk_hex = stored.split("$")
    dk = hashlib.pbkdf2_hmac("sha256", password.encode(),
                             bytes.fromhex(salt_hex), int(
iters))
    return hmac.compare_digest(dk.hex(), dk_hex)

record = hash_password("CorrectHorseBattery9!")
print("Stored:", record[:40], "...")
print("Correct password :", verify_password("
CorrectHorseBattery9!", record))
print("Wrong password   :", verify_password("password123",
record))
```

Listing 23: Salted, stretched password hashing with PBKDF2

Observe that the salt is stored *alongside* the hash in plain view – salts are not secrets. Their entire job is to be *unique*, so that an attacker must attack each password individually instead of testing one guess against the whole database at once. The iteration count is stored too, so the system can raise the work factor for new passwords without invalidating existing ones.

Task 3: Modern password hashing with passlib

```
from passlib.hash import bcrypt, argon2

b = bcrypt.using(rounds=12).hash("CorrectHorseBattery9!")
print("bcrypt :", b)
print("verify :", bcrypt.verify("CorrectHorseBattery9!", b
))

a = argon2.using(memory_cost=65536, time_cost=3).hash("
CorrectHorseBattery9!")
print("argon2 :", a)
print("verify :", argon2.verify("CorrectHorseBattery9!", a
))
```

Listing 24: Using bcrypt and argon2 through passlib

Both algorithms handle salting automatically and embed the salt and parameters in the output string, so you cannot accidentally forget them. Argon2 is the current recommendation because it is **memory-hard**: it deliberately requires a large amount of RAM per guess, which blunts the enormous parallelism advantage that GPU and ASIC hardware gives an attacker against purely CPU-bound schemes like PBKDF2.

Task 4: Measuring the work factor

```
import time
from passlib.hash import bcrypt

for rounds in [8, 10, 12, 14]:
    start = time.perf_counter()
    bcrypt.using(rounds=rounds).hash("test-password")
    elapsed = time.perf_counter() - start
    print(f"rounds={rounds:2d}   time={elapsed:.3f}s")
```

Listing 25: How cost parameters affect verification time

Each increment of the `rounds` parameter *doubles* the work, so the timings should roughly double at every step. The engineering trade-off is explicit: choose the highest cost your server can tolerate at peak login volume – commonly tuned to around 250–500 ms per verification. A delay invisible to a legitimate user multiplies an offline attacker’s total cost by the same factor.

Task 5: Password strength and entropy

```
import math, string

def entropy_bits(pw):
    pool = 0
    if any(c.islower() for c in pw): pool += 26
    if any(c.isupper() for c in pw): pool += 26
    if any(c.isdigit() for c in pw): pool += 10
    if any(c in string.punctuation for c in pw): pool += 32
    return len(pw) * math.log2(pool) if pool else 0

for pw in ["password", "Passw0rd!", "correct-horse-battery-staple-77"]:
    bits = entropy_bits(pw)
    verdict = "weak" if bits < 50 else "reasonable" if bits < 80 else "strong"
    print(f"{pw:35s} {bits:6.1f} bits ({verdict})")
```

Listing 26: Estimating password entropy and giving feedback

Notice that the long passphrase scores far higher than the short “complex” password despite being easier to remember and type – **length dominates character-set complexity** in this formula, which is why modern guidance (NIST SP 800-63B) favours long passphrases, forbids mandatory periodic rotation, and requires screening against known-breached password lists. Be aware this formula is an upper bound only: it assumes randomness, so a dictionary word remains weak no matter what the arithmetic says.

Summary:

This week you learned why plaintext and fast-hash storage both fail, implemented salted key-stretched hashing with PBKDF2, used bcrypt and Argon2 through `passlib`, tuned the work factor empirically, and estimated password entropy. The rule to remember: **never invent your own password storage – use Argon2 or bcrypt with a sensible cost, and let the library manage salts.**

8 Week 8: Authentication, Access Control, and Authorisation

Objective: Distinguish authentication from authorisation, implement multi-factor authentication, and model access control policies.

Tasks:

1. **Task 1:** Distinguish identification, authentication, and authorisation, and list the three authentication factors.
2. **Task 2:** Implement TOTP-based two-factor authentication.
3. **Task 3:** Implement a role-based access control (RBAC) check.
4. **Task 4:** Compare DAC, MAC, RBAC, and ABAC models.
5. **Task 5:** Apply least privilege and separation of duties to a scenario.

Details of Lab Experiment:

Task 1: Identification, authentication, authorisation

Identification is claiming who you are (entering a username). **Authentication** is proving that claim. **Authorisation** is deciding what the now-proven identity is permitted to do. Confusing the last two is a common and dangerous design error: an application that checks *whether* you are logged in but not *whether this particular user may access this particular record* has authentication without authorisation – the flaw class known as broken access control, which you will meet again in Week 12.

The three classical authentication factors are: something you **know** (password, PIN), something you **have** (phone, hardware token, smart card), and something you **are** (fingerprint, face, iris). True multi-factor authentication requires factors from *different* categories – a password plus a security question is still just one factor twice over.

Task 2: TOTP two-factor authentication

```
import pyotp, time

# --- Enrolment: generate a per-user secret and a
#       provisioning URI ---
secret = pyotp.random_base32()
uri = pyotp.totp.TOTP(secret).provisioning_uri(
    name="ali@example.edu.pk", issuer_name="InfoSec Lab")
print("Secret:", secret)
```



```

print("Enrol via QR code from URI:", uri)

# --- Verification at login ---
totp = pyotp.TOTP(secret)
current = totp.now()
print("Current code:", current)
print("Accepted      :", totp.verify(current))

time.sleep(31)
print("Same code after 31s:", totp.verify(current))    #
expired

```

Listing 27: Time-based one-time password enrolment and verification

TOTP derives a six-digit code from an HMAC of the shared secret and the current 30-second time window, so the code changes continuously and both sides compute it independently without any network traffic. Because it expires quickly, a code captured by an attacker is useless within seconds – but note that TOTP does *not* protect against real-time phishing, where a fake login page relays the code immediately; defending against that requires origin-bound factors such as FIDO2/WebAuthn security keys.

Task 3: Role-based access control

```

ROLE_PERMISSIONS = {
    "student": {"view_own_grades", "submit_assignment"},
    "instructor": {"view_own_grades", "view_class_grades",
                  "enter_grades"},
    "registrar": {"view_class_grades", "enter_grades", "
                  modify_transcript"},
}

USERS = {"ali": "student", "dr_khan": "instructor", "
         office1": "registrar"}

def is_allowed(user, permission):
    role = USERS.get(user)
    if role is None:
        return False                                # deny by
    default
    return permission in ROLE_PERMISSIONS.get(role, set())

for user in ["ali", "dr_khan", "office1", "unknown"]:
    print(f"{user:9s} enter_grades: {is_allowed(user, '
    enter_grades')}")

```

Listing 28: A minimal RBAC permission check

The most important line here is the `return False` for an unknown user: access control must **fail closed**, denying by default and permitting only

what is explicitly granted. A system that grants access when it cannot determine the answer – or that crashes into an "allow" path on error – inverts the entire security model.

Task 4: Access control models

- **DAC (Discretionary):** the resource owner decides who gets access, as with UNIX file permissions or sharing a Google Doc. Flexible, but privileges spread informally and are hard to audit.
- **MAC (Mandatory):** a central policy based on labels and clearances decides, and users cannot override it – used in military and high-assurance systems (SELinux, Bell–LaPadula).
- **RBAC (Role-Based):** permissions attach to roles and users are assigned roles, as in Task 3. This is the dominant enterprise model because it scales with organisational structure rather than with user count.
- **ABAC (Attribute-Based):** decisions evaluate attributes of user, resource, action, and context – "a nurse may read a record *if* assigned to that ward *and* on shift." Most expressive, most complex to administer.

Task 5: Least privilege and separation of duties

Least privilege means every user, process, and service receives only the minimum access needed for its function, and only for as long as needed – a web application should connect to its database with an account that cannot drop tables, and an administrator should use an unprivileged account for routine work. **Separation of duties** means no single individual controls an entire sensitive process: in the RBAC table above, notice that the person who *enters* grades and the person who *modifies transcripts* could deliberately be different roles, so that altering a final record requires collusion rather than one compromised account.

Apply both to a scenario and write it up: for a university fee-payment system, identify who should be able to create a fee voucher, who should approve it, who should record payment, and justify why those must not all be the same role.

Summary:

This week you separated authentication from authorisation, implemented TOTP-based MFA and an RBAC check that fails closed, compared the four access control models, and applied least privilege and separation of duties.

Broken access control is consistently ranked the #1 web application security risk – these concepts are not theoretical.

9 Week 9: Network Security Fundamentals and Traffic Analysis

Objective: Observe network traffic in the isolated lab, understand what unencrypted protocols expose, and see how TLS changes the picture.

Reminder: every activity in this lab is performed *only* between your own virtual machines on the host-only network. Capturing traffic on a network you do not own is unlawful.

Tasks:

1. **Task 1:** Review the TCP/IP layers and identify where each security control operates.
2. **Task 2:** Capture traffic between your own VMs with Wireshark and inspect a plaintext protocol.
3. **Task 3:** Repeat the capture over TLS and compare what remains visible.
4. **Task 4:** Inspect an X.509 certificate and explain the chain of trust.
5. **Task 5:** Explain ARP spoofing and man-in-the-middle risk conceptually, and identify the defences.

Details of Lab Experiment:

Task 1: Where security controls live

- **Link layer:** WPA3 for wireless; port security and 802.1X on switches.
- **Internet layer:** IPsec, firewall rules, network segmentation.
- **Transport layer:** TLS – by far the most widely deployed control, protecting HTTPS, secure email, and most APIs.
- **Application layer:** authentication, input validation, authorisation, application-level encryption.

Understanding the layering explains a frequent misconception: TLS protects data *in transit between endpoints*, but does nothing about a compromised endpoint, a malicious server, or data at rest – which is why “we use HTTPS” is a starting point for security, never a conclusion.

Task 2: Capturing plaintext traffic

On the target VM, run a simple HTTP service; on the analyst VM, capture the exchange.

```
# On the TARGET VM: serve a directory over plain HTTP (
    port 8000)
python3 -m http.server 8000

# On the ANALYST VM: capture only traffic to the target,
    on the lab interface
sudo tcpdump -i vboxnet0 host 192.168.56.101 -w
    lab_capture.pcap

# Generate traffic, then open the capture in Wireshark:
curl http://192.168.56.101:8000/
wireshark lab_capture.pcap
```

Listing 29: Capturing and filtering traffic between your own lab VMs

In Wireshark, apply the display filter `http` and use *Follow* → *TCP Stream*. You will see the complete request and response as readable text – URLs, headers, cookies, and any form data. The lesson is immediate and visceral: on a plaintext protocol, anyone positioned on the network path (a rogue access point, a compromised router, another device on the same shared Wi-Fi) sees everything.

Task 3: The same traffic over TLS

```
# On the TARGET VM: create a self-signed certificate for
    the lab only
openssl req -x509 -newkey rsa:2048 -nodes -keyout key.pem
    \
        -out cert.pem -days 30 -subj "/CN=lab.local"

# Serve over TLS
openssl s_server -accept 8443 -cert cert.pem -key key.pem
    -www

# On the ANALYST VM: capture again, then connect
curl -k https://192.168.56.101:8443/
```

Listing 30: Serving over HTTPS and re-capturing

Re-open the capture and apply the filter `tls`. The handshake is visible, but the application data is now opaque ciphertext. Note carefully what *still* leaks even under TLS: the source and destination IP addresses, the port, the packet sizes and timings, and (in older handshakes) the requested server name in the SNI field. This is **metadata**, and traffic analysis of it

alone can reveal a great deal about who is communicating with whom – which is precisely the passive attack category from Week 1.

Task 4: X.509 certificates and the chain of trust

```
# Inspect the lab certificate you just generated
openssl x509 -in cert.pem -noout -text | head -25

# Inspect a real chain (subject, issuer, validity only)
openssl s_client -connect example.com:443 -servername
example.com < /dev/null \
| openssl x509 -noout -subject -issuer -dates
```

Listing 31: Inspecting a certificate and its chain

A certificate binds a public key to an identity, and that binding is vouched for by a Certificate Authority’s *signature* – the exact mechanism you implemented in Week 6. Your browser trusts a site because the site’s certificate is signed by an intermediate CA, which is signed by a root CA already present in the operating system’s trust store. This is why the self-signed lab certificate above required `curl -k` to bypass verification: nothing vouches for it, and in any real deployment that warning must never be clicked through.

Task 5: ARP spoofing and man-in-the-middle risk (conceptual)

ARP has no authentication: a host asks ”who has this IP?” and believes whatever answer arrives first. An attacker on the same local segment can therefore claim to be the gateway, causing traffic to be routed through their machine – the classic local-network man-in-the-middle position. You are *not* asked to perform this attack; understanding the mechanism is what lets you recognise and defend against it.

Defences, in increasing order of strength:

- **Detection:** tools such as `arpwatch` alert on MAC/IP binding changes; switch logs show duplicate-MAC events.
- **Network controls:** Dynamic ARP Inspection and DHCP snooping on managed switches; port security limiting MAC addresses per port; network segmentation to shrink the broadcast domain.
- **Cryptographic controls:** the strongest answer is to assume the local network is hostile and rely on end-to-end TLS with proper certificate validation, so that a man-in-the-middle sees only ciphertext and cannot present a trusted certificate.

Summary:

This week you captured traffic in your isolated lab, saw plaintext protocols expose everything and TLS conceal content while still leaking metadata, inspected certificates and the chain of trust, and studied local man-in-the-middle risk defensively. The practical takeaway: **assume the network is hostile and encrypt end to end.**

10 Week 10: Host Hardening, Firewalls, and Logging

Objective: Reduce the attack surface of a host, configure a host firewall with a default-deny policy, and enable logging that supports later investigation.

Tasks:

1. **Task 1:** Enumerate and reduce the attack surface of your lab VM.
2. **Task 2:** Configure a default-deny host firewall with explicit allow rules.
3. **Task 3:** Harden the SSH service configuration.
4. **Task 4:** Apply patch management and verify update status.
5. **Task 5:** Configure logging and review authentication events.

Details of Lab Experiment:

Task 1: Enumerating attack surface

```
# Which services are listening, on which ports, owned by
  which process?
sudo ss -tulpn

# Which services start automatically at boot?
systemctl list-unit-files --state=enabled --type=service

# Disable anything not required on this host
sudo systemctl disable --now cups.service
```

Listing 32: Listing listening services on your own VM

Every listening port is a potential entry point, and every enabled service is code that must be patched and monitored. The single most cost-effective hardening step is simply *removing* what is not needed – a service that is not installed cannot be exploited, cannot be misconfigured, and does not need a patch cycle.

Task 2: A default-deny firewall

```
sudo ufw default deny incoming      # deny everything
inbound by default
```



```
sudo ufw default allow outgoing

# Then explicitly permit only what this host must serve
sudo ufw allow from 192.168.56.0/24 to any port 22 proto
      tcp
sudo ufw allow 443/tcp

sudo ufw enable
sudo ufw status verbose
```

Listing 33: Configuring UFW with a default-deny inbound policy

The ordering here expresses the central principle: **deny by default, then allow by exception**. A firewall built the other way around – permitting everything and blocking known-bad – fails silently the moment a new service appears, because you can only block threats you have already thought of. Note also that the SSH rule is restricted to the lab subnet rather than opened to the world; scoping a rule to a source range is nearly free and dramatically reduces exposure.

Task 3: Hardening SSH

```
# Generate a key pair on the client, then install the
# public key on the server
ssh-keygen -t ed25519 -C "lab-analyst"
ssh-copy-id user@192.168.56.101

# On the server, in /etc/ssh/sshd_config:
# PermitRootLogin no
# PasswordAuthentication no
# PubkeyAuthentication yes
# AllowUsers labuser
# MaxAuthTries 3
sudo sshd -t && sudo systemctl reload sshd
```

Listing 34: Key SSH server hardening directives

Disabling password authentication in favour of keys eliminates the entire category of brute-force and credential-stuffing attacks against SSH in one step, because there is no password to guess. Always run `sshd -t` to validate the configuration *before* reloading, and keep an existing session open while testing – locking yourself out of a remote host with a bad SSH config is a rite of passage best avoided.

Task 4: Patch management

```
sudo apt update
```

```
apt list --upgradable                # review what
will change

sudo apt upgrade -y
sudo apt install unattended-upgrades # automate
security patches
sudo dpkg-reconfigure --priority=low unattended-upgrades
```

Listing 35: Checking and applying security updates

The overwhelming majority of successful real-world intrusions exploit vulnerabilities for which a patch was *already available*, often for months. Timely patching is unglamorous and consistently outperforms sophisticated detection tooling in actual risk reduction – which is why automated security updates on servers are now considered a baseline control rather than an optimisation.

Task 5: Logging and review

```
# Failed and successful authentication attempts
sudo journalctl -u ssh --since "1 hour ago"
sudo grep "Failed password" /var/log/auth.log | tail -20
sudo lastb | head                # failed logins
sudo last | head                 # successful logins

# Install fail2ban to throttle repeated failures
automatically
sudo apt install fail2ban && sudo systemctl enable --now
fail2ban
sudo fail2ban-client status sshd
```

Listing 36: Reviewing authentication events

Logs are what make an incident *investigable* – without them, you cannot answer when an intrusion began, what was accessed, or whether it is over, which are exactly the questions Week 14's incident response process demands. Two refinements matter in practice: forward logs to a separate host so an attacker who compromises a machine cannot simply erase the evidence, and ensure clocks are synchronised (NTP) so that events from different systems can be correlated on a single timeline.

Summary:

This week you reduced attack surface, built a default-deny firewall, hardened SSH to key-only authentication, automated patching, and configured logging with automated throttling of repeated failures. These unglamorous controls – patch, minimise, restrict, log – prevent more real incidents than any single advanced technology.

11 Week 11: Malware Concepts and Defensive Analysis

Objective: Classify malware types, understand infection and persistence mechanisms conceptually, and apply defensive detection and response techniques.

Scope note: this lab studies malware *defensively*. No malicious code is written, obtained, or executed. All exercises use benign test artefacts (such as the EICAR test string, which is a harmless standardised antivirus test file, not malware).

Tasks:

1. **Task 1:** Classify the major malware categories by propagation and payload.
2. **Task 2:** Verify that endpoint protection is working using the EICAR test file.
3. **Task 3:** Perform safe static triage of a suspicious file (hashes, type, strings).
4. **Task 4:** Identify common persistence locations on a host.
5. **Task 5:** Design a layered anti-malware defence and a ransomware-resilient backup strategy.

Details of Lab Experiment:

Task 1: Malware classification

- **Virus:** attaches to a host file and requires user action to spread.
- **Worm:** self-propagating across networks with no user action – historically the fastest-spreading category.
- **Trojan:** disguised as legitimate software; the user installs it willingly.
- **Ransomware:** encrypts data and demands payment – an availability and (increasingly, via data theft) confidentiality attack.
- **Spyware/keylogger:** covertly collects information – a confidentiality attack.
- **Rootkit:** subverts the operating system to hide its own presence, which is why detection from a *compromised* host is unreliable and offline or boot-media analysis is preferred.

- **Botnet client:** enrolls the host into an attacker-controlled network for further attacks.

Note that these categories describe *behaviour*, and modern samples routinely combine several – a trojan dropper that installs a rootkit-hidden keylogger and later deploys ransomware is one incident, not four.

Task 2: Verifying endpoint protection with EICAR

```
sudo apt install clamav && sudo freshclam      # install and
update signatures

# The EICAR test file is a harmless, industry-standard
scanner test --
# it is NOT malware and performs no action whatsoever.
printf 'X50!P%#@AP[4\\PZX54(P^)7CC)7}$EICAR-STANDARD-
ANTIVIRUS-TEST-FILE!$H+H*' \
> eicar_test.txt

clamscan eicar_test.txt      # expected: detection
reported
rm eicar_test.txt
```

Listing 37: Confirming the scanner works using the harmless EICAR test string

This verifies the detection pipeline end to end – scanner installed, signatures current, alerting functional – without ever handling real malicious code. Testing that a control *actually fires* is a habit worth generalising: an untested backup, alert, or scanner should be assumed non-functional until proven otherwise.

Task 3: Safe static triage

```
# NEVER execute an unknown file. Examine it statically
only.
sha256sum suspicious.bin      # hash for
reputation lookup
file suspicious.bin           # real type,
regardless of extension
strings -n 8 suspicious.bin | head -40  # embedded URLs,
paths, commands
exiftool suspicious.bin       # metadata
```

Listing 38: Static examination of a suspicious file without executing it

Static analysis inspects a file without running it, which is what makes it safe. The hash can be checked against threat-intelligence services; file

reveals type mismatches (an "invoice.pdf" that is actually an executable is an immediate red flag); and `strings` often exposes hard-coded domains or command-line fragments. Dynamic analysis – actually executing the sample – must only ever occur in a purpose-built, disposable, network-isolated sandbox, and is beyond the scope of this course.

Task 4: Persistence locations

```
crontab -l ; sudo crontab -l          # user and root
cron jobs
ls -la /etc/cron.* /etc/systemd/system/ # scheduled
tasks and unit files
systemctl list-unit-files --state=enabled
ls -la ~/.config/autostart/ 2>/dev/null # desktop
autostart entries
cat ~/.bashrc ~/.profile | tail -20    # shell startup
hooks
```

Listing 39: Auditing common autostart locations on Linux

Malware must survive a reboot to be useful, so it needs a **persistence mechanism** – and the number of such locations is finite and well documented, which is what makes systematic auditing effective. On Windows the equivalents are the Run/RunOnce registry keys, scheduled tasks, services, and startup folders. Knowing where to look converts an open-ended hunt into a checklist.

Task 5: Layered defence and ransomware resilience

Design and document a defence-in-depth strategy assuming that *any single control will eventually fail*:

- **Prevent:** patching, application allow-listing, disabling macros from the internet, email filtering, least privilege so malware inherits limited rights.
- **Detect:** endpoint detection and response, integrity monitoring, log correlation, alerting on the persistence locations from Task 4.
- **Respond:** isolate the host from the network, preserve evidence *before* wiping, rebuild from known-good media rather than "cleaning."
- **Recover:** the **3-2-1 backup rule** – three copies, on two different media types, with one kept offline or immutable. The offline copy is the specific control that defeats ransomware, because encryption of live systems cannot reach media that is not attached.

Critically, backups must be *restore-tested* on a schedule. An organisation that discovers its backups are unreadable during a ransomware incident is, in practical terms, an organisation with no backups at all.

Summary:

This week you classified malware, verified endpoint protection with a harmless test file, performed safe static triage, audited persistence locations, and designed a layered defence with ransomware-resilient backups. The defining assumption of modern defence: **plan for prevention to fail, and make detection and recovery good enough that failure is survivable.**

12 Week 12: Web Application Security and Secure Coding

Objective: Understand the most common web application vulnerability classes and, for each, implement the correct defensive coding pattern.

Tasks:

1. **Task 1:** Review the OWASP Top 10 vulnerability classes.
2. **Task 2:** Prevent SQL injection using parameterised queries.
3. **Task 3:** Prevent cross-site scripting through correct output encoding.
4. **Task 4:** Implement input validation with an allow-list and safe file path handling.
5. **Task 5:** Apply security headers and CSRF protection.

Details of Lab Experiment:

Task 1: The OWASP Top 10

The OWASP Top 10 is the industry's consensus list of the most critical web application risks. The recurring themes are: **broken access control** (consistently ranked first), **cryptographic failures**, **injection**, **insecure design**, **security misconfiguration**, **vulnerable components**, **identification and authentication failures**, **software and data integrity failures**, **logging and monitoring failures**, and **server-side request forgery**. Notice how many of these you have already addressed from other angles: access control in Week 8, cryptographic failures in Weeks 3–7, logging in Week 10.

Task 2: Preventing SQL injection

```
import sqlite3

conn = sqlite3.connect(":memory:")
conn.execute("CREATE TABLE users (id INTEGER, name TEXT,
    role TEXT)")
conn.execute("INSERT INTO users VALUES (1, 'ali', 'student
    ')")

# WRONG: string concatenation lets input change the query's
# STRUCTURE
```

```
# query = "SELECT * FROM users WHERE name = '" +
    user_input + "'"

# RIGHT: parameterised query -- input is always treated as
    DATA, never code
def find_user(conn, name):
    cur = conn.execute("SELECT id, name, role FROM users
        WHERE name = ?", (name,))
    return cur.fetchall()

print(find_user(conn, "ali"))
# A crafted input is now harmlessly treated as a literal
    username:
print(find_user(conn, "ali' OR '1'='1"))    # returns
    nothing
```

Listing 40: Parameterised queries – the correct defence against SQL injection

The root cause of every injection flaw is the same: untrusted input is concatenated into a string that is then *interpreted* as code. Parameterised queries (also called prepared statements) fix this structurally by sending the query template and the data to the database separately, so user input can never alter the query's meaning no matter what it contains. Note that escaping or filtering input is *not* an adequate substitute – it is error-prone and has been bypassed repeatedly; parameterisation removes the possibility rather than trying to sanitise it away.

Task 3: Preventing cross-site scripting

```
import html
from markupsafe import escape

user_comment = '<script>alert("stolen cookie")</script>'

# WRONG: inserting raw input into HTML lets the browser
    execute it
# page = "<div>" + user_comment + "</div>"

# RIGHT: encode on OUTPUT, for the context it is being
    placed into
safe = html.escape(user_comment)
print("<div>" + safe + "</div>")
# Renders as visible text: &lt;script&gt;alert(...)&lt;/
    script&gt;

# Template engines (Jinja2/Flask) autoescape by default --
    keep it enabled
print(escape(user_comment))
```

Listing 41: Context-aware output encoding to prevent XSS

XSS occurs when attacker-controlled data is delivered to another user's browser and executed as script, typically to steal session cookies or perform actions as the victim. The defence is **encoding at output time, appropriate to the context** – HTML body, HTML attribute, JavaScript, and URL contexts each require different encoding, which is why using a template engine's autoescaping (and never disabling it with constructs like `|safe`) is far more reliable than manual escaping. Setting cookies `HttpOnly` additionally prevents JavaScript from reading them, limiting the damage if an XSS flaw does slip through.

Task 4: Input validation and safe file paths

```
import os, re

# Validate against an ALLOW-list of what is permitted,
# not a DENY-list of what is forbidden.
def valid_username(name):
    return bool(re.fullmatch(r"[a-z0-9_]{3,20}", name))

print(valid_username("ali_khan"), valid_username("../../
etc/passwd"))

# Path traversal prevention: resolve, then confirm it
# stayed inside the root
UPLOAD_ROOT = os.path.realpath("./uploads")

def safe_path(filename):
    candidate = os.path.realpath(os.path.join(UPLOAD_ROOT,
    filename))
    if not candidate.startswith(UPLOAD_ROOT + os.sep):
        raise ValueError("Path traversal attempt blocked")
    return candidate

print(safe_path("report.pdf"))
try:
    safe_path("../../etc/passwd")
except ValueError as e:
    print("Blocked:", e)
```

Listing 42: Allow-list validation and path traversal prevention

Always validate with an **allow-list** – specify exactly what is acceptable and reject everything else – because a deny-list can only block the bad inputs you thought of in advance, and attackers specialise in the ones you did not. For file paths, the essential step is to *resolve* the path first (collapsing

.. components) and only then verify that the result still lies inside the intended directory; checking the raw input string for .. before resolution is a classic bypassable mistake.

Task 5: Security headers and CSRF protection

```
from flask import Flask, session, request, abort
import secrets

app = Flask(__name__)
app.secret_key = secrets.token_bytes(32)
app.config.update(SESSION_COOKIE_HTTPONLY=True,
                  SESSION_COOKIE_SECURE=True,
                  SESSION_COOKIE_SAMESITE="Lax")

@app.after_request
def set_security_headers(resp):
    resp.headers["Content-Security-Policy"] = "default-src 'self'"
    resp.headers["X-Content-Type-Options"] = "nosniff"
    resp.headers["X-Frame-Options"] = "DENY"
    resp.headers["Strict-Transport-Security"] = "max-age=31536000"
    return resp

def check_csrf():
    token = session.get("csrf_token")
    if not token or not secrets.compare_digest(
        token, request.form.get("csrf_token", "")):
        abort(403)
```

Listing 43: Applying security headers and CSRF tokens in Flask

Each header closes a specific attack path: **Content-Security-Policy** restricts which sources may execute scripts (defence in depth behind Task 3), **nosniff** stops the browser guessing content types, **X-Frame-Options** prevents clickjacking, and **HSTS** forces HTTPS. The CSRF token defends against a different problem entirely: an attacker's site causing the victim's *authenticated* browser to submit a request to your application. Because the attacker cannot read the token, requiring it – and comparing it in constant time, as in Week 6 – proves the request originated from your own page.

Summary:

This week you worked through the major web vulnerability classes and implemented the correct defence for each: parameterised queries, context-aware output encoding, allow-list validation with safe path resolution, and security headers with CSRF tokens. The unifying principle across all of

them is to **separate data from code, and validate against what is allowed rather than what is forbidden.**

13 Week 13: Steganography, Data Protection, and Digital Forensics Basics

Objective: Understand data hiding, protect data at rest, and apply sound evidence-handling practice in a basic forensic examination.

Tasks:

1. **Task 1:** Implement LSB steganography and explain how it differs from encryption.
2. **Task 2:** Detect hidden data and explain steganalysis at a high level.
3. **Task 3:** Encrypt data at rest with a passphrase-derived key.
4. **Task 4:** Apply forensic evidence-handling principles and compute evidence hashes.
5. **Task 5:** Extract and interpret file metadata, and understand secure deletion.

Details of Lab Experiment:

Task 1: Least-significant-bit steganography

```
from PIL import Image

def hide(image_path, message, out_path):
    img = Image.open(image_path).convert("RGB")
    bits = "".join(f"{ord(c):08b}" for c in message + "\0")
    pixels = list(img.getdata())
    if len(bits) > len(pixels) * 3:
        raise ValueError("Message too large for this image")
    out, i = [], 0
    for (r, g, b) in pixels:
        chan = [r, g, b]
        for k in range(3):
            if i < len(bits):
                chan[k] = (chan[k] & 0xFE) | int(bits[i]);
            i += 1
        out.append(tuple(chan))
    img.putdata(out); img.save(out_path)

def reveal(image_path):
    bits = ""
```

```
for (r, g, b) in Image.open(image_path).convert("RGB")
    .getdata():
    bits += str(r & 1) + str(g & 1) + str(b & 1)
chars = [chr(int(bits[i:i+8], 2)) for i in range(0,
len(bits) - 8, 8)]
msg = "".join(chars)
return msg.split("\0")[0]

hide("cover.png", "Meet at 6pm", "stego.png")
print("Recovered:", reveal("stego.png"))
```

Listing 44: Hiding a message in the low bits of image pixels

Changing the least significant bit of a colour channel alters its value by at most 1 out of 255 – imperceptible to the eye, which is what makes the hiding effective. Note the critical conceptual distinction: **steganography hides the *existence* of a message; cryptography hides its *content***. Steganography alone provides no protection if the hiding scheme is discovered, which is why serious use always encrypts the payload *before* embedding it. Note also that the output must be a lossless format (PNG, BMP); saving as JPEG destroys the hidden bits.

Task 2: Detecting hidden data

```
# File size anomalies and format inconsistencies
ls -l cover.png stego.png
file stego.png
exiftool stego.png

# Appended-data check: content after the image's logical
end
binwalk stego.png
```

Listing 45: Basic indicators of steganographic content

Steganalysis looks for statistical deviations from what a natural image should look like – LSB embedding, for example, flattens the distribution of least significant bits in a way that chi-square tests can detect. In forensic practice, the simpler indicators matter just as much: an unexpected file size, a mismatch between extension and actual type, or data appended after the file's structural end. Perfect undetectability is much harder to achieve than the simple embedding in Task 1 suggests.

Task 3: Encrypting data at rest

```
import os, base64
from cryptography.hazmat.primitives.kdf.pbkdf2 import
PBKDF2HMAC
```

```
from cryptography.hazmat.primitives import hashes
from cryptography.fernet import Fernet

def key_from_passphrase(passphrase, salt):
    kdf = PBKDF2HMAC(algorithm=hashes.SHA256(), length=32,
                      salt=salt, iterations=600_000)
    return base64.urlsafe_b64encode(kdf.derive(passphrase.
        encode()))

salt = os.urandom(16)                                # store
                                                    # alongside the ciphertext
f = Fernet(key_from_passphrase("lab-passphrase", salt))

with open("secret.txt", "rb") as src:
    token = f.encrypt(src.read())
with open("secret.enc", "wb") as dst:
    dst.write(salt + token)                            # salt is not
                                                    # secret

with open("secret.enc", "rb") as src:
    blob = src.read()
f2 = Fernet(key_from_passphrase("lab-passphrase", blob
    [:16]))
print("Decrypted:", f2.decrypt(blob[16:])[40:])
```

Listing 46: Passphrase-derived key encryption of a file

A passphrase is not a key – it has far too little entropy and the wrong length – so a **key derivation function** stretches it into one, using the same salting and high iteration count you applied to passwords in Week 7 and for the same reason: to make offline guessing expensive. At the operating system level, full-disk encryption (LUKS, BitLocker, FileVault) applies the same idea to an entire volume and should be standard on every laptop.

Task 4: Forensic evidence handling

```
# Work only on a COPY; hash before and after to prove
# nothing changed
sha256sum evidence.img > evidence.sha256

# Create a working copy (real cases use a hardware write-
# blocker)
dd if=evidence.img of=working_copy.img bs=4M status=
progress

sha256sum working_copy.img
sha256sum -c evidence.sha256                # verify the original is
unaltered
```

Listing 47: Imaging and hashing evidence to preserve integrity

Three principles govern all forensic work. **Never analyse the original** – work on a verified copy, and use a write-blocker so the source cannot be modified. **Prove integrity with hashes** computed before and after acquisition, exactly the mechanism from Week 5. **Maintain chain of custody** – a documented record of who held the evidence, when, and why – because evidence whose handling cannot be accounted for may be inadmissible regardless of what it contains.

Task 5: Metadata and secure deletion

```
# Metadata often reveals far more than the file's visible
content
exiftool photo.jpg | grep -Ei "date|gps|make|model|
software|author"

# Strip metadata before publishing a document or image
exiftool -all= -overwrite_original photo.jpg

# Secure deletion (note: ineffective on SSDs and
journaling filesystems)
shred -vzf -n 3 sensitive.txt
```

Listing 48: Extracting metadata and removing it before sharing

Metadata leakage is a genuine and frequently underestimated privacy failure – photographs commonly embed GPS coordinates and device identifiers, and office documents retain author names and revision history. On secure deletion, be precise about the limits: overwriting tools like **shred** assume the filesystem writes in place, which is false for SSDs (wear leveling relocates writes) and journaling filesystems. On modern storage, the reliable answers are full-disk encryption from the outset – so discarding the key renders the data unrecoverable – or the drive's own secure-erase command.

Summary:

This week you implemented and detected LSB steganography, encrypted data at rest with a passphrase-derived key, applied forensic imaging and hashing with chain-of-custody discipline, and examined metadata leakage and the real limits of secure deletion. Keep the core distinction sharp: **steganography hides existence, cryptography hides content** – and only the latter survives discovery.

14 Week 14: Risk Management, Security Policy, and Incident Response

Objective: Assess and treat risk systematically, write enforceable policy, and follow a structured incident response process – then integrate the whole course into a final project.

Tasks:

1. **Task 1:** Identify assets and threats, and build a risk register.
2. **Task 2:** Quantify risk and select an appropriate treatment for each entry.
3. **Task 3:** Draft an enforceable security policy section.
4. **Task 4:** Walk through the incident response lifecycle on a scenario.
5. **Task 5:** Complete the final project: a full security assessment of a small system.

Details of Lab Experiment:

Task 1: Asset identification and the risk register

Begin with assets, because you cannot protect what you have not enumerated: data (student records, research data, credentials), systems (servers, laptops, network devices), people, and reputation. For each asset, identify plausible threats and existing vulnerabilities, then record them in a structured register.

```
risks = [  
  {"asset": "Student records DB", "threat": "SQL  
injection",  
   "likelihood": 3, "impact": 5, "control": "Parameterised queries + WAF"},  
  {"asset": "Staff laptops", "threat": "Theft / loss",  
   "likelihood": 4, "impact": 4, "control": "Full-disk  
encryption"},  
  {"asset": "Email accounts", "threat": "Phishing /  
credential theft",  
   "likelihood": 5, "impact": 4, "control": "MFA + user  
training"},  
  {"asset": "Web server", "threat": "Unpatched CVE  
exploitation",  
   "likelihood": 3, "impact": 5, "control": "Automated  
patching"},  
]
```



```
for r in sorted(risks, key=lambda x: -(x["likelihood"] * x
["impact"])):
    score = r["likelihood"] * r["impact"]
    level = "CRITICAL" if score >= 20 else "HIGH" if score
    >= 12 else "MEDIUM"
    print(f"{score:2d} {level:8s} {r['asset']:22s} {r['
    threat']}'})")
```

Listing 49: A minimal risk register in Python

Task 2: Quantifying and treating risk

Scoring likelihood and impact on a 1–5 scale and multiplying gives a defensible ordering – its real value is not numerical precision but forcing an explicit, reviewable judgement and directing finite budget at the highest-scoring items first. Every risk must then receive one of four treatments:

- **Mitigate:** apply controls to reduce likelihood or impact (patching, MFA, encryption). The default choice.
- **Transfer:** shift financial consequence to another party (cyber insurance, contractual terms with a provider). Note that reputational damage and regulatory liability generally cannot be transferred.
- **Avoid:** stop doing the risky activity (decommission a legacy service, stop collecting data you do not need). Often the cheapest and most overlooked option.
- **Accept:** formally acknowledge and document a risk whose treatment costs more than its expected loss – but acceptance must be an explicit, signed decision by someone with authority, never a silent default arising from inaction.

Task 3: Writing enforceable policy

An effective policy is specific, measurable, and assigns responsibility – “staff should use strong passwords” is unenforceable, whereas the following is auditable:

4.2 Authentication Requirements. All accounts accessing university systems shall be protected by a passphrase of at least 12 characters, screened against a list of known-breached credentials at the time of setting. Multi-factor authentication is mandatory for all staff accounts and for any account with administrative privileges. Shared accounts are prohibited. Credentials shall not be reused across university and personal services. *Owner:* IT Security Manager. *Review:* annually. *Non-compliance:* account suspension pending review.

Note the structure: a stated requirement, a named owner, a review cycle, and a defined consequence. A policy document without these last three elements is an aspiration, not a control, and will not survive an audit.

Task 4: The incident response lifecycle

Work through this scenario as a written exercise: *a staff member reports that files on a shared drive have been renamed with an unfamiliar extension and a ransom note has appeared.*

1. **Preparation:** the work done *before* anything happens – contact lists, defined roles, tested offline backups, logging enabled, a written plan. Almost all incident outcomes are determined at this stage.
2. **Detection and Analysis:** confirm it is a genuine incident, determine scope (which hosts, which data, which accounts), and establish an initial timeline from logs.
3. **Containment:** isolate affected hosts from the network – short-term containment first to stop spread, then a considered longer-term approach. Preserve volatile evidence *before* powering anything off.
4. **Eradication:** remove the cause – rebuild from known-good media rather than attempting to clean, reset all potentially exposed credentials, close the entry vector.
5. **Recovery:** restore from verified offline backups, monitor closely for recurrence, and return to production in a controlled, staged manner.
6. **Lessons Learned:** conduct a blameless post-incident review within two weeks, and feed concrete changes back into Preparation – this feedback loop is what makes the cycle a cycle rather than a checklist.

Document, for each stage, exactly who decides, who acts, and who must be notified – including whether the incident triggers a legal or regulatory reporting obligation.

Task 5: Final project

Conduct and document a complete security assessment of a small system of your choosing (a course project web application, a home network, or a departmental service you are authorised to assess). Your report must include:

1. **Scope and authorisation:** exactly what is in scope, and written evidence that you are permitted to assess it.

2. **Asset inventory and threat model:** what is being protected, from whom, and what the impact of compromise would be.
3. **Cryptographic review:** algorithms, modes, key management, and credential storage assessed against Weeks 3–7.
4. **Configuration and network review:** attack surface, firewall posture, patch status, TLS configuration, and logging, per Weeks 9–10.
5. **Application review:** access control and the secure coding patterns of Week 12.
6. **Risk register:** findings scored and prioritised as in Tasks 1–2.
7. **Remediation plan:** specific, actionable recommendations with owners and a realistic timeline, distinguishing quick wins from structural changes.

Write the report for two audiences at once: an executive summary that a non-technical decision maker can act on, and technical detail sufficient for an engineer to reproduce and fix each finding. That dual-audience skill is, in professional practice, as valuable as the technical work itself.

Summary:

In this final week you built a risk register, applied the four risk treatments, wrote enforceable policy, worked through the six-stage incident response lifecycle, and specified the capstone assessment that integrates the entire course. Congratulations on completing every core domain of information security this course set out to cover – cryptography, authentication and access control, network and host security, malware defence, secure coding, forensics, and governance. Carry forward the habit that underlies all of it: **assume compromise is possible, defend in layers, and verify that your controls actually work.**